

Course code and name:	F20DP – Distributed and Parallel Technologies – 2024 - 2025
Type of assessment:	Pair
Coursework Title:	F20DP Coursework2
Student Name:	Godugu Anil Himam
Student ID Number:	H00447039

Declaration of authorship. By signing this form:

- **I declare** that the work I have submitted for individual assessment OR the work I have contributed to a group assessment, is entirely my own. I have NOT taken the ideas, writings or inventions of another person and used these as if they were my own. My submission or my contribution to a group submission is expressed in my own words. Any uses made within this work of the ideas, writings or inventions of others, or of any existing sources of information (books, journals, websites, etc.) are properly acknowledged and listed in the references and/or acknowledgements section.
- I confirm that I have read, understood and followed the Universitys Regulations on plagiarism as published on the [Universitys website](#), and that I am aware of the penalties that I will face should I not adhere to the University Regulations.
- I confirm that I have read, understood and avoided the different types of plagiarism explained in the University guidance on [Academic Integrity and Plagiarism](#)

Student Signature (*type your name*): Godugu Anil Himam

Date: 04/04/2025

Copy this page and insert it into your coursework file in front of your title page.
For group assessment each group member must sign a separate form and all forms must be included with the group submission.

Course code and name:	F20DP – Distributed and Parallel Technologies – 2024 - 2025
Type of assessment:	Pair
Coursework Title:	F20DP Coursework2
Student Name:	PORCHER Jeanne
Student ID Number:	H00494480

Declaration of authorship. By signing this form:

- **I declare** that the work I have submitted for individual assessment OR the work I have contributed to a group assessment, is entirely my own. I have NOT taken the ideas, writings or inventions of another person and used these as if they were my own. My submission or my contribution to a group submission is expressed in my own words. Any uses made within this work of the ideas, writings or inventions of others, or of any existing sources of information (books, journals, websites, etc.) are properly acknowledged and listed in the references and/or acknowledgements section.
- I confirm that I have read, understood and followed the Universitys Regulations on plagiarism as published on the [Universitys website](#), and that I am aware of the penalties that I will face should I not adhere to the University Regulations.
- I confirm that I have read, understood and avoided the different types of plagiarism explained in the University guidance on [Academic Integrity and Plagiarism](#)

Student Signature (*type your name*): PORCHER Jeanne

Date: 04/04/2025

Copy this page and insert it into your coursework file in front of your title page.

For group assessment each group member must sign a separate form and all forms must be included with the group submission.



Coursework 2: Sum Totient in SYCL and C++MPI

Porcher Jeanne | Godugu Anil

Module: F20DP Distributed and Parallel Technologies.

Deadline: 04/04/2025

Lecturers: Dr Rob Stewart | Dr Hans-Wolfgang Loidl

1. Section 1: Introduction

The aim of this second coursework, like the first one, is to parallelise the calculation of the sum of the calculations of the Euler totient over a range of integer values. This corresponds, for an integer N , to the number of positive integers up to N and relatively prime to N . To do this, we will use 2 implementations:

- SYCL
- C+MPI

The goal is then to compare the sequential and parallel performance of our implementations on both CPUs and GPUs of the Robotarium cluster to determine whether parallelism is necessary and, if so, which method seems most appropriate to the Euler totient algorithm.

To do this, we will first use the *gprof* tool on the sequential version as well of each version of the SYCL code to establish the areas to be parallelized. We will then use the following metrics to check the performance of parallelism and possibly apply performance optimisation:

- Runtime measurements
- Speedups
- Efficiency

The benchmarks are executed on an HPC cluster using Slurm. The script runs each parallelised version with a fixed 64 CPU cores per task and measuring execution time across three different input sizes (15000, 30000, 100000). The results are stored in csv files, with each configuration being tested five times for accuracy to analyse the parallel speedup and computational efficiency.

1.1: Statement of authorship

Godugu Anil: Responsible for the implementation and evaluation of MPI.

Porcher Jeanne: Responsible for the implementation and evaluation of SYCL, Introduction and Section 2.

2. Section 2: Sequential Performance Measurements

To get an initial overview of the problem, we analysed the behaviour of the sequential executable, which will perform the totientRange on the [1; 15000] interval.

% Time	Cumulative Seconds	Self Seconds	Calls	Self Time per Call	Total Time per Call (s)	Function Name
86.83	4.38	4.38	112507500	0	0	hcf(unsigned int, unsigned int)
9.03	4.83	0.46	15000	0	0	euler(unsigned int)
3.57	5.01	0.18	112507500	0	0	relprime(unsigned int, unsigned int)
0.6	5.04	0.03				cl::sycl::detail::computeCppMagicKernel(cl::sycl::queue&::{lambda(cl::sycl::handler&)#1}::operator()(cl::sycl::handler&) const::{lambda(cl::sycl::item<1, true>)#1}::operator()(cl::sycl::item) const
0	5.04	0	3	0	0	std::chrono::duration<long, std::ratio<1l, 1000000000l> >::count() const
0	5.04	0	2	0	0	std::chrono::time_point<std::chrono::_V2::system_clock, std::chrono::duration<long, std::ratio<1l, 1000000000l> > >::time_since_epoch() const
0	5.04	0	1	0	0	_GLOBAL_sub_I_Z3hcfjj
0	5.04	0	1	0	5.01	sumTotient(unsigned int, unsigned int)

Table 1: Extract of the gprof command results on the sequential version of the Euler Totient Algorithm

Running *gprof* shows that the *hcf* function is responsible for 86.83% of the total execution time, with an extremely high number of calls of 112,507,500. This suggests that it is the main performance bottleneck and that it is called excessively, within nested loops. This makes it an ideal target for optimisation.

Next, the *euler* function is another major hotspot, contributing 9.03% of the execution time over 15,000 calls. This function is invoked by *sumTotient*, which in turn is the main function called from *main*. Given that *sumTotient* executes only once but is responsible for most of the execution time (including its children), the efficiency of its subroutines is crucial.

We can therefore deduce that the method for calculating the largest common factor is a source of slowdown, and algorithmic optimisation of this function could lead to substantial performance gains. Additionally, cache performance should be considered. Since *hcf* is called millions of times, ensuring that frequently accessed data remains in cache would improve performance. Hence, a more efficient approach, such as using memorisation to store previously calculated values or getting rid of the brute force aspect of *hcf*, could significantly reduce redundant calculations.

As seen in the first coursework, the profiling analysis indicates that *hcf* (and therefore *relprime*) is the critical function to optimise, followed by *euler*. However, we decided to leave it as is to outline the GPU implication in the runtime improvement of the calculation. The aim of this study will be to determine, between OpenMP and Haskell, which aspect of the latter should be prioritised: performance or productivity.

To thoroughly evaluate the performance of our parallelisation, we will analyse the runtime of the sequential implementation alongside the C+MPI and SYCL versions, comparing their efficiency and scalability.

We will run 5 times on each of those inputs:

- DS1: calculating the sum of totients between 1 and 15000.
- DS2: calculating the sum of totients between 1 and 30000.
- DS3: calculating the sum of totients between 1 and 100000.

Following those inputs, the sequential results are:

Input	Run1	Run2	Run3	Run4	Run5	Median
15000	10.937	10.875	10.931	10.924	10.931	10.931
30000	46.389	46.462	46.352	46.399	46.462	46.399
100000	567.344	566.626	567.566	567.361	566.841	567.344

Table 2: Sequential runtime of the original C sum totient algorithm (expressed in seconds)

3. Section 3: C+MPI Parallel Performance Measurements

C+MPI is a parallel programming model that combines the C language with the MPI (Message Passing Interface) framework to enable communication between several processes running on separate machines or nodes, often in a high-performance computing environment. The main objective of C+MPI is to facilitate the parallelization of distributed algorithms by dividing the work between different processes, while optimizing memory management and inter-process communication.

(a) Section 3a: **Runtime Table**

Input	Cores	Run1	Run2	Run3	Median	Median - fast	Median-slow
15000	1	13.45	13.42	13.53	13.45	0.03	-0.08
15000	2	6.49	5.27	5.26	5.27	0.01	-1.22
15000	4	1.84	1.84	1.84	1.84	0	0
15000	8	0.84	0.8	0.81	0.81	0.01	-0.03
15000	12	0.57	0.59	0.56	0.57	0.01	-0.02
15000	16	0.5	0.48	0.41	0.48	0.07	-0.02
15000	20	0.39	0.38	0.39	0.39	0.01	0
15000	24	0.4	0.41	0.36	0.4	0.04	-0.01
15000	28	0.38	0.35	0.36	0.36	0.01	-0.02
15000	32	0.4	0.36	0.31	0.36	0.05	-0.04
15000	36	0.42	0.35	0.38	0.38	0.03	-0.04
15000	40	0.34	0.32	0.36	0.34	0.02	-0.02
15000	44	0.32	0.32	0.37	0.32	0	-0.05
15000	48	0.37	0.4	0.47	0.4	0.03	-0.07
15000	52	0.36	0.38	0.42	0.38	0.02	-0.04
15000	56	0.42	0.42	0.46	0.42	0	-0.04
15000	60	0.41	0.35	0.39	0.39	0.04	-0.02
15000	64	0.4	0.44	0.32	0.4	0.08	-0.04
15000	68	0.61	0.77	0.63	0.63	0.02	-0.14
15000	72	0.69	0.7	0.65	0.69	0.04	-0.01
15000	76	0.76	0.77	0.71	0.76	0.05	-0.01
15000	80	0.7	0.67	0.73	0.7	0.03	-0.03
15000	84	0.89	0.8	0.85	0.85	0.05	-0.04
15000	88	0.71	0.67	0.78	0.71	0.04	-0.07
15000	92	0.73	0.77	0.69	0.73	0.04	-0.04
15000	96	0.87	0.95	0.64	0.87	0.23	-0.08
15000	100	0.51	0.44	0.94	0.51	0.07	-0.43
15000	104	0.41	0.37	0.49	0.41	0.04	-0.08
15000	108	0.74	0.88	0.87	0.87	0.13	-0.01
15000	112	0.65	0.68	0.78	0.68	0.03	-0.1
15000	116	0.66	0.7	0.74	0.7	0.04	-0.04
15000	120	0.85	0.78	0.73	0.78	0.05	-0.07
15000	124	0.74	0.63	0.79	0.74	0.11	-0.05
15000	128	0.87	0.73	0.71	0.73	0.02	-0.14
15000	132	0.83	1.04	0.86	0.86	0.03	-0.18
15000	136	0.8	0.67	0.89	0.8	0.13	-0.09

15000	140	0.82	0.86	0.76	0.82	0.06	-0.04
15000	144	0.86	0.83	0.8	0.83	0.03	-0.03
15000	148	0.82	0.78	0.76	0.78	0.02	-0.04
15000	152	0.85	0.89	0.84	0.85	0.01	-0.04
15000	156	1.08	0.95	0.8	0.95	0.15	-0.13
15000	160	0.97	1.02	0.94	0.97	0.03	-0.05
15000	164	0.87	1.19	1.05	1.05	0.18	-0.14
15000	168	0.97	0.86	0.93	0.93	0.07	-0.04
15000	172	1.08	1.03	1.13	1.08	0.05	-0.05
15000	176	1.13	1.04	1	1.04	0.04	-0.09
15000	180	1.03	0.95	1.02	1.02	0.07	-0.01
15000	184	1.12	1.12	1.04	1.12	0.08	0
15000	188	1.1	0.72	1.04	1.04	0.32	-0.06
15000	192	1.11	1.14	1.07	1.11	0.04	-0.03

Table 8: Parallel runtime of the Euler totient algorithm between 1 and 15000 using C+MPI (expressed seconds)

The runtime table for DS1 reveals that the Euler Totient sum calculation benefits from an increase in the number of cores, but with diminishing returns beyond a certain point. At 1 core, the median runtime is the highest (~13.45ss), with notable improvements in performance as the number of cores increases. The optimum core utilisation range is between 4 and 12 cores, where the median runtime drops to around 0.5-1.8ms, offering the best performance for this input size (15,000).

Above 16 cores, the performance improvement peaks, and at higher core counts (e.g. 68 and 132 cores), the execution time becomes erratic, with some executions being even slower than with fewer cores. This indicates a potential parallelism overload or inefficiencies in the use of additional cores. The differences between the fastest and slowest executions remain small for lower numbers of cores, but increase for higher numbers of cores, suggesting that scalability issues arise in the event of excessive parallelism. Using around 4 to 12 cores seems then to offer a good balance between performance and resource efficiency, while adding more cores leads to less perceptible or even negative improvements.

Input	Cores	Run1	Run2	Run3	Median	Median - fast	Median-slow
30000	1	51.8	51.92	52.02	51.92	0.12	-0.1
30000	2	22.04	22.05	22.03	22.04	0.01	-0.01
30000	4	7.58	7.62	7.66	7.62	0.04	-0.04
30000	8	3.35	3.33	3.3	3.33	0.03	-0.02
30000	12	2.15	2.15	2.15	2.15	0	0
30000	16	1.61	1.62	1.66	1.62	0.01	-0.04
30000	20	1.34	1.32	1.34	1.34	0.02	0
30000	24	1.14	1.15	1.14	1.14	0	-0.01
30000	28	1.06	1.08	1.03	1.06	0.03	-0.02
30000	32	0.93	0.93	0.97	0.93	0	-0.04
30000	36	0.93	0.91	0.91	0.91	0	-0.02
30000	40	0.88	0.82	0.9	0.88	0.06	-0.02

30000	44	0.82	0.83	0.96	0.83	0.01	-0.13
30000	48	0.83	0.83	0.8	0.83	0.03	0
30000	52	0.85	0.88	0.84	0.85	0.01	-0.03
30000	56	0.71	0.82	0.74	0.74	0.03	-0.08
30000	60	0.77	0.74	0.82	0.77	0.03	-0.05
30000	64	0.74	0.69	0.77	0.74	0.05	-0.03
30000	68	0.66	0.7	0.73	0.7	0.04	-0.03
30000	72	0.64	0.65	0.69	0.65	0.01	-0.04
30000	76	0.64	0.73	0.65	0.65	0.01	-0.08
30000	80	0.76	0.74	0.8	0.76	0.02	-0.04
30000	84	0.7	0.62	0.64	0.64	0.02	-0.06
30000	88	0.93	0.71	0.65	0.71	0.06	-0.22
30000	92	0.64	0.69	0.77	0.69	0.05	-0.08
30000	96	1.01	0.98	0.95	0.98	0.03	-0.03
30000	100	0.62	0.69	0.93	0.69	0.07	-0.24
30000	104	1.02	0.66	0.64	0.66	0.02	-0.36
30000	108	0.58	0.67	0.59	0.59	0.01	-0.08
30000	112	0.91	1	1.17	1	0.09	-0.17
30000	116	1.16	0.62	0.69	0.69	0.07	-0.47
30000	120	1.15	0.67	0.83	0.83	0.16	-0.32
30000	124	0.95	0.58	0.74	0.74	0.16	-0.21
30000	128	0.97	0.97	0.92	0.97	0.05	0
30000	132	0.63	0.69	0.97	0.69	0.06	-0.28
30000	136	0.57	0.77	1.05	0.77	0.2	-0.28
30000	140	0.72	0.61	0.66	0.66	0.05	-0.06
30000	144	0.75	0.77	0.65	0.75	0.1	-0.02
30000	148	0.75	0.77	1.04	0.77	0.02	-0.27
30000	152	1.1	1.17	0.76	1.1	0.34	-0.07
30000	156	0.72	0.76	0.73	0.73	0.01	-0.03
30000	160	0.65	0.7	0.8	0.7	0.05	-0.1
30000	164	0.74	0.8	0.8	0.8	0.06	0
30000	168	0.7	0.72	0.88	0.72	0.02	-0.16
30000	172	1.12	1.21	1.15	1.15	0.03	-0.06
30000	176	1.2	1.16	1.17	1.17	0.01	-0.03
30000	180	1.21	1.3	1.14	1.21	0.07	-0.09
30000	184	1.2	1.2	1.28	1.2	0	-0.08
30000	188	1.16	1.23	1.35	1.23	0.07	-0.12
30000	192	1.19	1.35	1.8	1.35	0.16	-0.45

Table 9: Parallel runtime of the Euler totient algorithm between 1 and 30000 using C+MPI (expressed seconds)

The runtime table for DS2 reveals a similar trend to that of the 15,000 input, where performance improves with increasing core count, but plateaus or even deteriorates at higher core counts. Initially, at 1 core, the median execution time is the highest (~51.92s). As more cores are added, the median execution time decreases significantly, with optimal performance between 4 and 12 cores, where median times range from 1.61s to 7.62s.

However, when the number of cores exceeds 12, the performance improvement becomes less pronounced. From 20 cores upwards, there are fluctuations in execution time, with some instances being even slower than faster executions at lower core counts, indicating diminishing returns. When the number of cores exceeds 100, runtime variability increases, and some configurations even exceed the runtime of the 1-core configuration.

Input	Cores	Run1	Run2	Run3	Median	Median - fast	Median-slow
100000	1	566.72	565.48	566.32	566.32	0.84	-0.4
100000	2	269.91	268.73	269.04	269.04	0.31	-0.87
100000	4	92.23	91.56	92.78	92.23	0.67	-0.55
100000	8	43.7	42.09	40.07	42.09	2.02	-1.61
100000	12	29.51	28.81	28.91	28.91	0.1	-0.6
100000	16	20.84	19.04	21.7	20.84	1.8	-0.86
100000	20	14.94	17.15	17.05	17.05	2.11	-0.1
100000	24	13.65	12.57	14.44	13.65	1.08	-0.79
100000	28	11.47	11.39	12.44	11.47	0.08	-0.97
100000	32	10.28	9.68	10.55	10.28	0.6	-0.27
100000	36	8.91	8.69	9	8.91	0.22	-0.09
100000	40	8.75	8.73	8.31	8.73	0.42	-0.02
100000	44	7.49	7.21	7.53	7.49	0.28	-0.04
100000	48	6.87	6.91	7.13	6.91	0.04	-0.22
100000	52	6.6	6.77	6.81	6.77	0.17	-0.04
100000	56	6.24	6.24	6.31	6.24	0	-0.07
100000	60	5.77	5.85	5.86	5.85	0.08	-0.01
100000	64	5.63	5.62	5.59	5.62	0.03	-0.01
100000	68	5.23	5.43	5.38	5.38	0.15	-0.05
100000	72	5.04	4.9	4.94	4.94	0.04	-0.1
100000	76	4.59	4.56	4.74	4.59	0.03	-0.15
100000	80	4.5	4.49	4.53	4.5	0.01	-0.03
100000	84	4.25	4.34	4.27	4.27	0.02	-0.07
100000	88	4.07	4.09	4.22	4.09	0.02	-0.13
100000	92	4.1	4.1	4.14	4.1	0	-0.04
100000	96	3.9	3.92	3.91	3.91	0.01	-0.01
100000	100	3.7	3.65	3.73	3.7	0.05	-0.03
100000	104	3.73	3.66	3.73	3.73	0.07	0
100000	108	3.62	3.59	3.64	3.62	0.03	-0.02
100000	112	3.67	3.43	3.43	3.43	0	-0.24
100000	116	3.28	3.55	3.39	3.39	0.11	-0.16
100000	120	3.25	3.15	3.27	3.25	0.1	-0.02
100000	124	3.06	3.23	3.12	3.12	0.06	-0.11
100000	128	3.07	3.09	3.1	3.09	0.02	-0.01
100000	132	3	3.14	3.13	3.13	0.13	-0.01
100000	136	3	3.07	2.99	3	0.01	-0.07
100000	140	2.9	2.85	2.96	2.9	0.05	-0.06
100000	144	2.86	2.81	2.72	2.81	0.09	-0.05

100000	148	2.71	2.69	2.74	2.71	0.02	-0.03
100000	152	2.78	2.8	3.06	2.8	0.02	-0.26
100000	156	2.71	2.62	2.78	2.71	0.09	-0.07
100000	160	2.67	2.66	2.62	2.66	0.04	-0.01
100000	164	2.57	2.64	2.57	2.57	0	-0.07
100000	168	2.64	2.62	2.6	2.62	0.02	-0.02
100000	172	2.52	2.53	2.6	2.53	0.01	-0.07
100000	176	2.49	2.53	2.5	2.5	0.01	-0.03
100000	180	2.53	2.53	2.46	2.53	0.07	0
100000	184	2.67	2.9	2.57	2.67	0.1	-0.23
100000	188	2.53	2.47	2.5	2.5	0.03	-0.03
100000	192	2.49	2.35	2.56	2.49	0.14	-0.07

Table 10: Parallel runtime of the Euler totient algorithm between 1 and 100000 using C+MPI (expressed seconds)

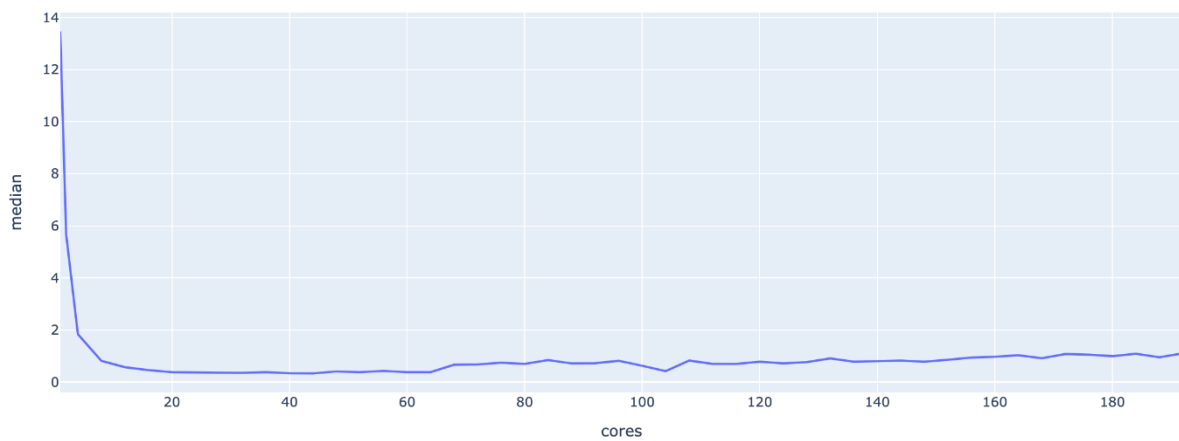
The runtime table for DS3 follows a similar pattern to that of the previous input sizes, showing significant improvements in performance as cores are used, with diminishing returns at higher numbers of cores. With one core, the median execution time is 566.32s, decreasing progressively as the number of cores increases. Optimal performance is achieved between 40 and 80 cores, where median execution times are between 3.7s and 4.5s.

Beyond 80 cores, runtimes stabilise, with only slight improvements. When the number of cores exceeds 100, performance fluctuations become more noticeable, with no significant reduction in execution time despite the increase in the number of cores. For example, median times at 100 cores and above hover around 3.7s, with slight performance increases or decreases observed at higher core counts.

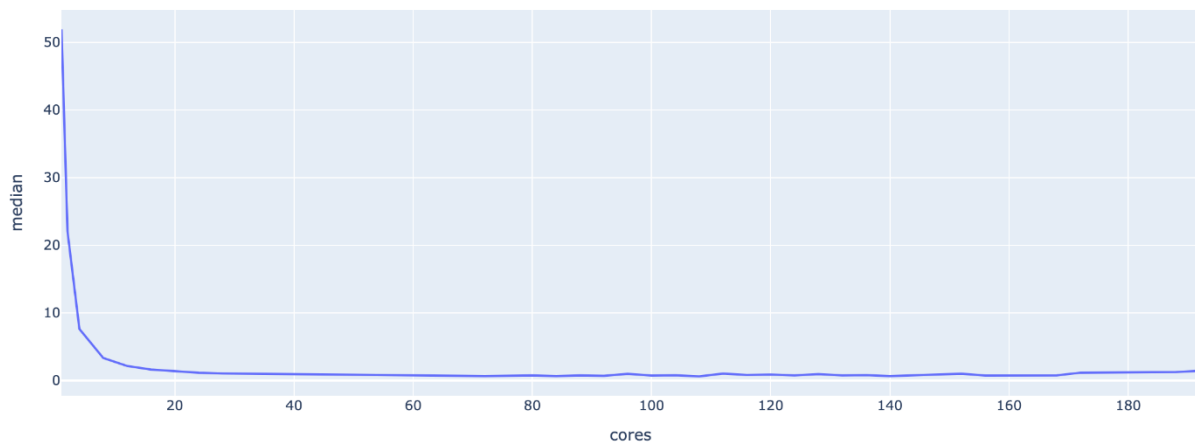
(b) Section 3b: **Runtime Graphs**

In the following section the runtime graphs for the execution of the Euler Totient program are displayed individually each input size. The x-axis represents the scaling of the program in cores while the y-axis represents the median execution time of the program for the corresponding input size.

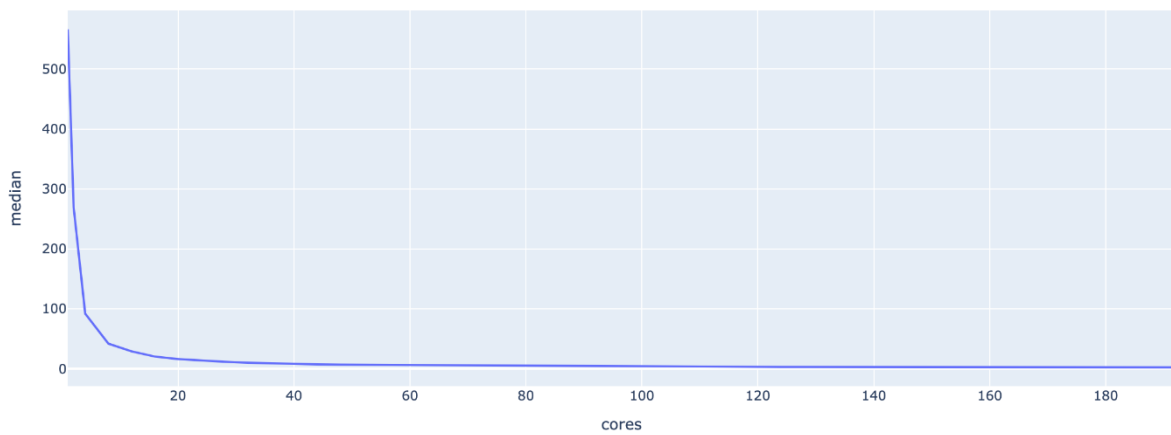
Runtime at 15000



Runtime at 30000



Runtime at 100000

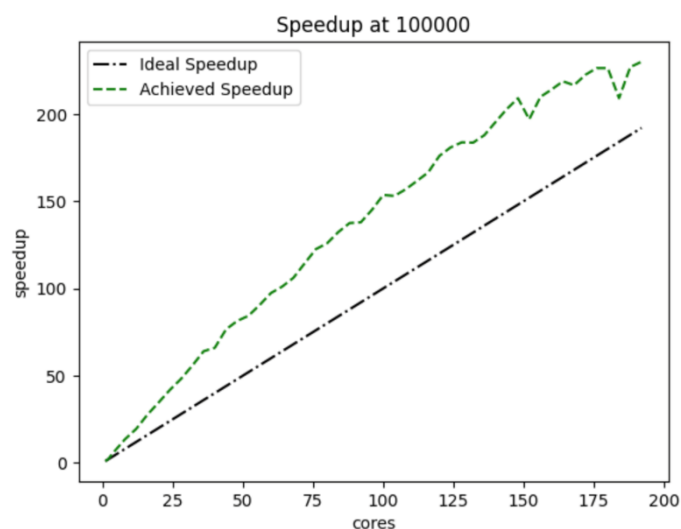
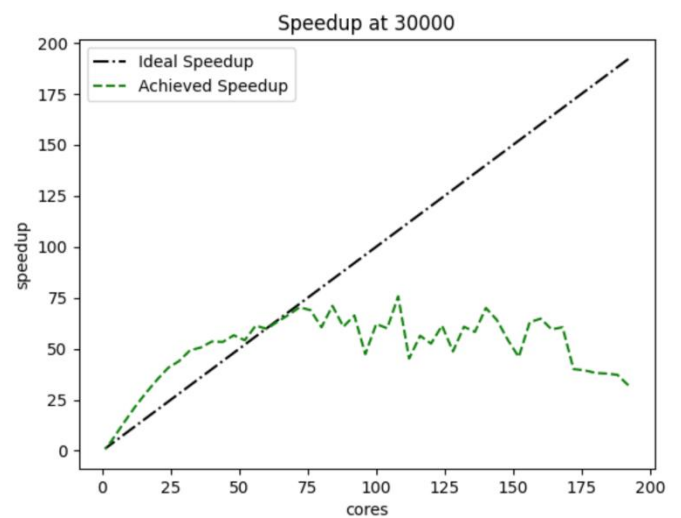
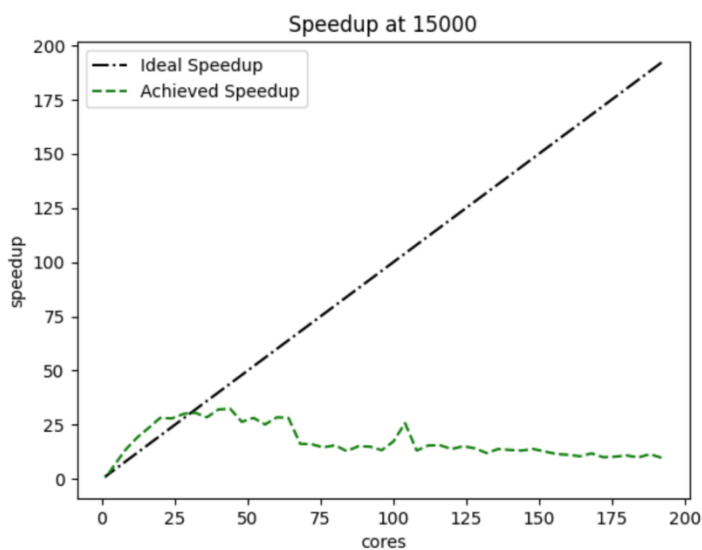


The runtime performance illustrated in the above figures shows that parallelism is helping in computation by increasing the efficiency of computing as the number of

cores increase. Each of the graphs track a smooth elbow curve depicting the exponential improvements made in the calculation speed by applying parallelism to solve the problem. There is approximately a 15x reduction in the calculation time at 8 cores for each input size as illustrated by the images. After which the program makes limited gains in calculation speed hitting diminishing returns.

(c) Section 3c: **Speedups**

In the following section graphs of the speedup achieved by the program are illustrated during the execution of the Euler Totient program. The speedup was calculated by considering the execution time of the sequential program in section 2. as the benchmark. The x-axis represents the scaling of the program in cores while the y-axis represents the speedup achieved by the program during execution.

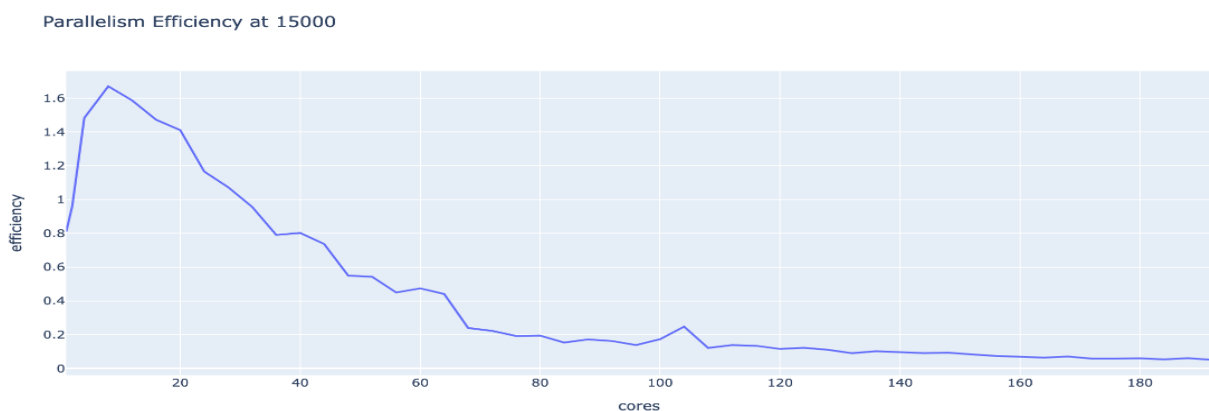


The above graphs visualise the speedup achieved by the program with increasing scale of computing cores. At input sizes 15000 and 30000 respectively the program managed to trace the ideal speedup line up to 25 cores and 75 cores, after which the program couldn't speedup any further by providing consistent speed till the termination of the program. At an input size of 15000 the program topped 26x the speed of the sequential program after which it provided a consistent speed of approximately 23x the sequential program. Similarly, at an input size of 30000 the program achieved a top speed of 75x the sequential program.

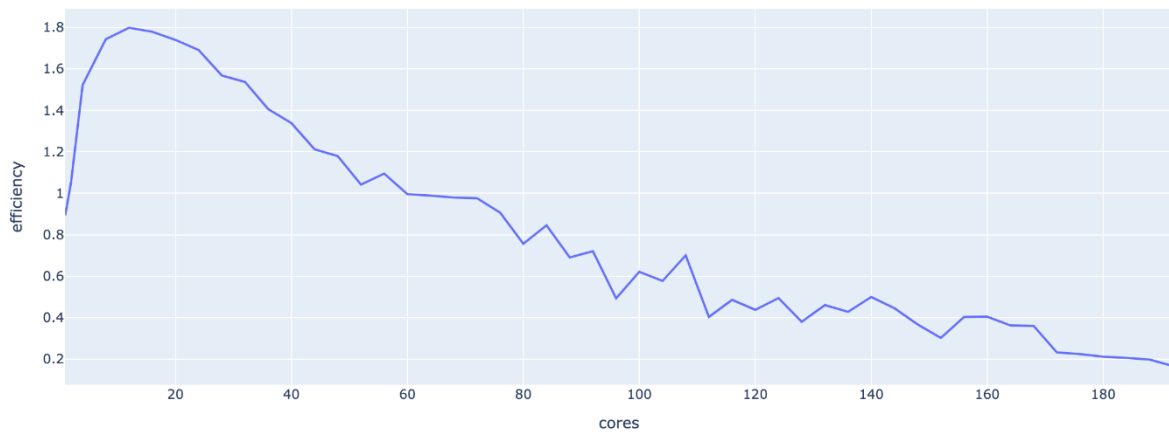
In case of the program execution with an input size of 10000 the program managed consistently achieve the ideal speedup or better during the entire experiment. This can be attributed to the embarrassingly parallel paradigm of programming as introduced in this course. Due to the higher input size the program was able to consistently split tasks among several cores at scales performing extremely fast.

(d) Section 3d: **Parallelism Efficiency**

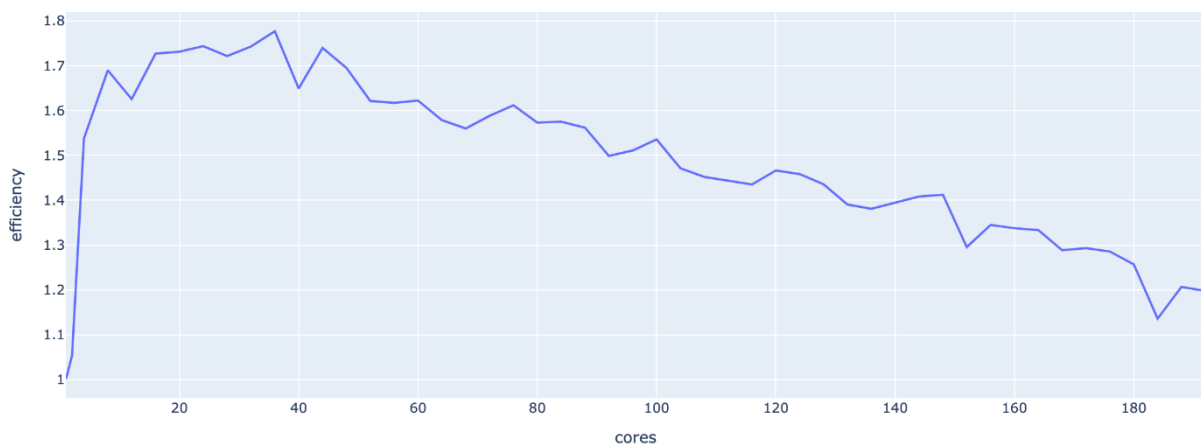
In the following section the parallelism efficiency achieved by the program are illustrated during the execution of the Euler Totient program. The parallelism efficiency was grounded around the speedup achieved by the program for different input sizes as analysed in section 3c. The x-axis represents the scaling of the program in cores while the y-axis represents the parallelism efficiency achieved by the program during execution.



Parallelism Efficiency at 30000



Parallelism Efficiency at 100000



The above graphs visualise the parallelism efficiency achieved by the program with increasing scale of computing cores. At input sizes 15000 and 30000 respectively the program managed to achieve extremely efficient parallelism for up to 20 cores and 37 cores respectively. After which the overhead of assigning tasks for multiple cores for small input sizes drastically reduced the efficiency of the program.

The parallelism efficiency of the program was much better for the input size of 100000 providing the embarrassingly parallel paradigm of programming a better representation of the achievable performance. The overhead of splitting the calculation among several cores was subsidised by the speedup provided by the program. The parallelism efficiency was sustained for longer time in this case as the number of calculations grew exponentially. However, towards the end of the program the efficiency decreased due to high overhead across the cores that needed synchronisation to accumulate the result.

4. Section 4: SYCL Parallel Performance Measurements

SYCL is a parallel programming model based on OpenCL, enabling tasks to be executed on devices such as CPUs, GPUs and FPGAs. Its aim is to simplify the parallelisation of algorithms by offering a standard C++ interface, while optimizing the use of available hardware resources. SYCL makes it possible to efficiently parallelise complex algorithms, by automatically managing memory and tasks on a variety of architectures.

(a) Section 4a: SYCL version 1

This first version is a naive usage of SYCL performing only the storage of each $\Phi(i)$ value in a SYCL array of length of the different inputs then summing those on the host CPU with a classic *for* loop.

As seen in *Table 1*, the C execution is inherently sequential which explains the large runtimes. Whereas in the SYCL version of the algorithm, each work-item in the kernel independently executes the calculation of the totient function for a specific number N . The *hcf* and *relprime* functions are being mapped as lambda functions within the kernel which allows for direct customization of how data is accessed or computed in parallel. This makes the kernel more adaptable and efficient since it can leverage the underlying architecture's features more effectively.

The main difference here is that the parallel version doesn't check all i values sequentially. Instead, each work-item checks the coprimeness for its corresponding value N in parallel.

In addition, the use of buffers and accessors in the SYCL version enables memory to be managed efficiently. Unlike the sequential version, which directly manipulates arrays in memory, the SYCL version abstracts from manual memory management, simplifying implementation while ensuring optimised data management between the host and the device. This abstraction also reduces the potential errors associated with explicit memory transfers and improves code maintainability.

Overall, the main improvement brought by the SYCL version lies in the parallelism and optimisation of memory management, which makes it possible to achieve significant performance gains compared with the sequential C version, especially when working with large value ranges:

Input	Run1	Run2	Run3	Run4	Run5	Median
15000	423	479	411	419	480	423
30000	526	544	591	523	539	539
100000	2202	2060	2086	2199	2067	2086

Table 3: Parallel GPU runtime of the first version of the Euler totient algorithm using SYCL (expressed in milliseconds)

If we look at the results of the *gprof* command for this first version on the GPU, we observe that the time repartition as well as the number of calls to each function is the same or very similar to those seen in *Table 1*, but SYCL handles them separately as lambda functions.

% Time	Cumulative Seconds	Self Seconds	Calls	Self Time per Call (s)	Total Time per Call (s)	Function Name
88.77	5.65	5.65	112507500	0.05	0.05	main::{lambda(ct::sycl::handler&)#1)::operator()(ct::sycl::handler&) const::{lambda(ct::sycl::id<1>)#1)::operator()(ct::sycl::id const::{lambda(unsigned int, unsigned int)#1)::operator()(unsigned int, unsigned int) const
6.76	6.08	0.43	15000	28.67	422.73	main::{lambda(ct::sycl::handler&)#1)::operator()(ct::sycl::handler&) const::{lambda(ct::sycl::id<1>)#1)::operator()(ct::sycl::id const
4.17	6.34	0.27	112507500	0	0.05	main::{lambda(ct::sycl::handler&)#1)::operator()(ct::sycl::handler&) const::{lambda(ct::sycl::id<1>)#1)::operator()(ct::sycl::id const::{lambda(unsigned int, unsigned int)#2)::operator()(unsigned int, unsigned int) const
0.31	6.36	0.02				sumTotient(unsigned int, unsigned int)

Table 4: Extract of the gprof command results on the first SYCL version of the Euler Totient Algorithm

Indeed, Table 4 shows that the number of calls of *hcf*, *relprime* and *euler* are the same as the sequential version but the most interesting part is the “Total Time per Call (s)” columns that shows that the *relprime* function is now where all the runtime is happening (422ms out of a 423ms median) instead of *sumTotient* for the sequential version.

(b) Section 4b: SYCL version 2: Reduction variable

Version 2 of the algorithm uses a SYCL reduction on the GPU device to perform the sum of $\Phi(i)$ values more efficiently. Unlike version 1, which simply stores the values in a SYCL array on the GPU and sums them on the CPU using a conventional loop, version 2 performs the sum directly on the GPU using SYCL reduction variables, reducing the need for communication between the GPU and CPU and improving the overall performance of the algorithm.

Input	Run1	Run2	Run3	Run4	Run5	Median	Speedup
15000	437	501	439	479	480	479	22.82
30000	597	607	601	605	597	601	77.20
100000	2113	2108	2106	2108	2111	2108	269.14

Table 5: Parallel GPU runtime of the second version of the Euler totient algorithm using SYCL (expressed in milliseconds)

Using this second version, we get results pretty similar to the first SYCL version (Table 3) which may raise questions about the effectiveness of using a reduction variable but, as we’ll see later, this method requires finding the right group size. Though, we can still observe a massive speedup compared to the sequential C version.

As shown by Table 5, speedups increase strongly with the size of the input going as far as 269x. We chose to plot those to outline the parallelism efficiency of SYCL and the usage of a GPU compared to a CPU which we will touch in more detail later.

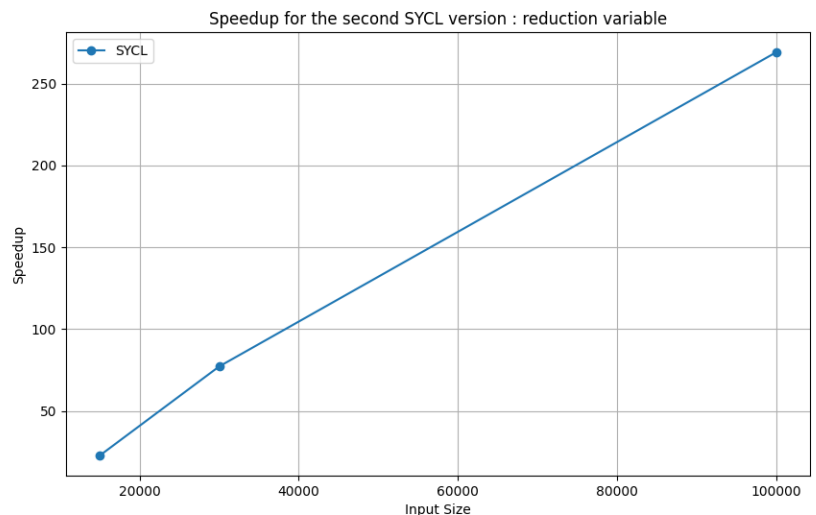


Figure 1: Speedup of the second version of the Euler totient algorithm using SYCL

(c) Section 4c: **SYCL version 3: Varying group sizes**

Version 3 of the algorithm improves on version 2 by adding explicit management of the size of the workgroup (WG) on the GPU. Unlike version 1, where the sum of $\Phi(i)$ values was performed on the CPU after storage in a SYCL array on the GPU, and version 2, which uses a reduction on the GPU to perform this sum more efficiently, version 3 goes further by specifying the size of the workgroup when the SYCL kernel is executed.

This workgroup (WG) size must be an exact divisor of the ND range size, which optimises the distribution of computations across the GPU's computing units. Here, we will test the following configurations [4, 8, 16, 32, 64, 128, 160] on the Euler totient computation between 1 and 400000, making it possible to assess the impact of the size of the workgroup on performance. As a result, version 3 offers additional flexibility and finer optimisation of the use of GPU resources than previous versions.

Input	WorkGroup	Run1	Run2	Run3	Run4	Run5	Median	Speedup
400000	4	349072	349061	348925	352066	349063	349063	34.96
400000	8	188469	188562	188546	188596	188480	188546	64.73
400000	16	101450	101261	101448	101338	101351	101351	120.42
400000	32	54553	54241	54427	54243	54381	54381	224.43
400000	64	30550	33588	30551	30484	30550	30550	399.51
400000	128	27583	27641	27490	27648	27588	27588	442.40
400000	160	27946	27796	27845	27880	28143	27880	437.77
Device								
400000	(version 2)	27023	27024	26918	27012	27020	27020	451.70
400000	Sequential	12425460	12204900	12262289	11987002	12015530	12204900	1

Table 6: Parallel GPU runtime of the third and second versions of the Euler totient algorithm using SYCL between 1 and 400000 using different work sizes (expressed in milliseconds)

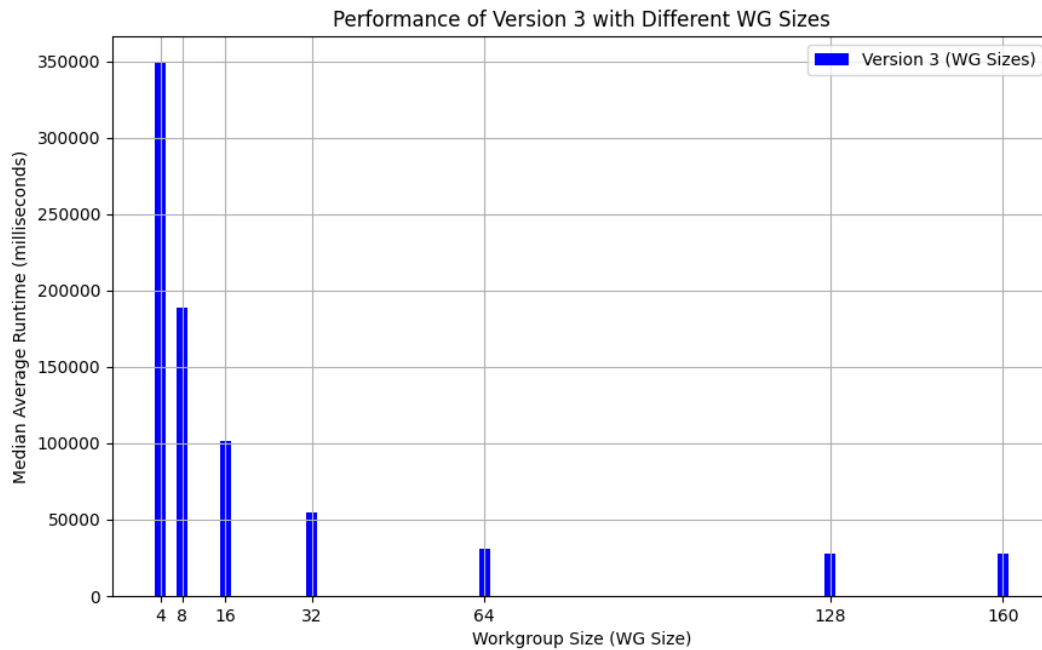


Figure 2: Parallel GPU runtime of the third version of the Euler totient algorithm using SYCL between 1 and 400000 using different work sizes

As shown in *figure 2* and *Table 6*, the best runtime of the totient calculation is reached with a workgroup size of **128** for the third SYCL version with a runtime of only **27.6 seconds** and a speedup of **442.5** compared to the sequential version which takes around **3.4 hours** to complete. Indeed, the ideal workgroup size is decided by the GPU used. Here, we use the NVidia K20 GPUs which have 13 compute units, each with 192 cores resulting in $192 \times 13 =$ **2496 cores**. In theory, to have the most efficient runtime in the $[1, 400000]$ totient range, we then need to find a workgroup size such as $400000 / WG = 2496$ so that each unit is responsible for one workgroup.

In theory, among the group sizes tested, $WG = 160$ should be the best as:

- $400,000 / 4 = 100,000$
- $400,000 / 8 = 50,000$
- $400,000 / 16 = 25,000$
- $400,000 / 32 = 12,500$
- $400,000 / 64 = 6250$
- $400,000 / 128 = 3125$
- **$400,000 / 160 = 2500$**

In practice, as we previously saw, $WG = 128$ has the best runtime but only by **292ms** compared to $WG = 160$ as if the group size division threshold was between these 2 values.

On the other hand, we have the second SYCL version algorithm which uses a group size decided by the SYCL device driver for the NVidia K20 GPU which has a runtime of **27.02 seconds** and the best speedup: **451.7**. We can then conclude that, at least for an algorithm as simple as the Euler's totient range calculation, the device driver is best suited to finding the best group size and therefore the best execution time.

(d) Section 4d: **Compare CPU and GPU performance of SYCL**

As seen previously, GPU performances are way better than CPU's. In order to get a better understanding of how striking the difference is, we plotted a grouped bar chart showing the runtimes on the CPU and GPU.

Input	Run1	Run2	Run3	Run4	Run5	Median
15000	4744	4814	4873	5101	5025	4873
30000	9146	8428	9399	9246	9251	9246
100000	35021	35038	35301	34689	33903	35021

Table 7: Parallel CPU runtime of the second version of the Euler Totient algorithm using SYCL (expressed in milliseconds)

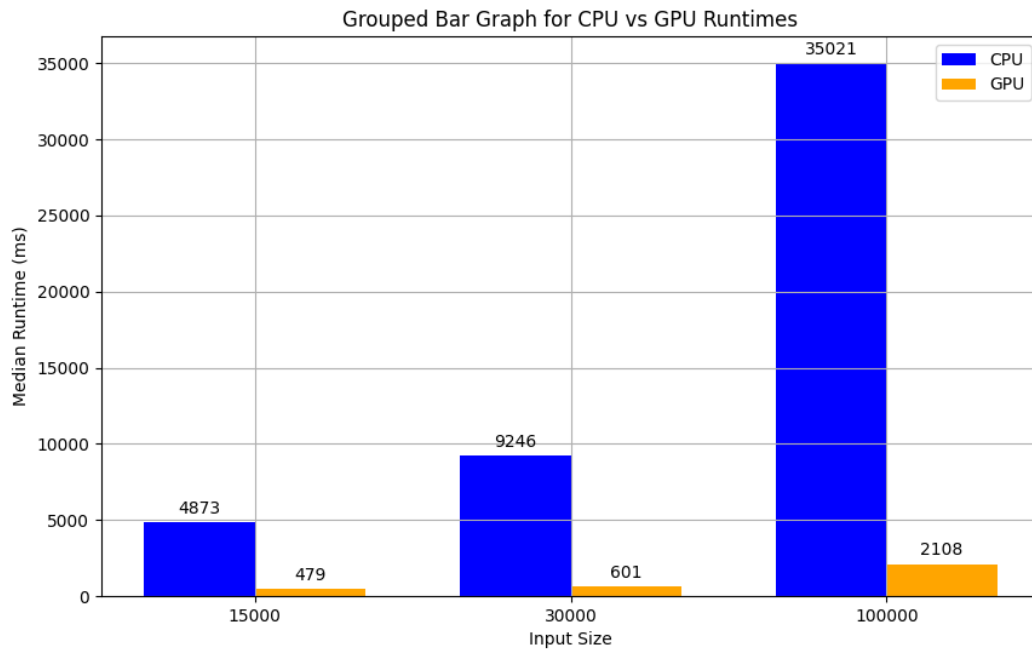


Figure 3: Parallel GPU vs CPU runtimes of the second version of the Euler totient algorithm

Based on the results shown in Table 7 and Figure 3, we obtain the following speedups:

- DS1: GPU is **10.17** times faster than the CPU
- DS2: GPU is **15.38** times faster than the CPU
- DS3: GPU is **16.61** times faster than the CPU

We can then observe that the GPU efficiency increases with the problem size which coincides with the results obtained in section 4c with a way bigger size problem of 400,000.

SYCL is designed as a portable programming model, allowing code to run on multiple parallel architectures such as CPUs, GPUs and FPGAs without requiring any modifications. This portability is advantageous because it ignores hardware-specific details, allowing developers to focus on algorithmic implementation rather than device-specific optimisations. In addition, SYCL integrates seamlessly with the standard C++ language, making it

accessible without the need to adopt a new language paradigm. It also simplifies memory management and task execution, ensuring compatibility between different architectures while reducing the potential errors associated with manual memory management. The results presented in the previous sections confirm that the same implementation of SYCL runs successfully on both AMD CPUs and NVidia GPUs, demonstrating the framework's ability to maintain functional portability between devices.

However, our analysis of the performance results highlights significant disparities between CPU and GPU execution times. Each of the previous sections show that for an input size of 15,000, the GPU executes the computation 10.17 times faster than the CPU. For larger inputs, such as 30,000 and 100,000, the performance gap widens further, with the GPU being 15.38 and 16.61 times faster respectively. These results confirm that GPUs, which are optimised for parallel workloads, exploit SYCL's capabilities more effectively than CPUs, which are inherently better suited to sequential or moderately parallel tasks.

Although SYCL enables successful execution on different architectures, the extent to which performance is maintained varies significantly. The tests show that although the same SYCL program can be run on both a 64-core AMD CPU and a 2496-core NVidia GPU, the GPU benefits significantly more from SYCL's parallel execution model. *Figure 3*, comparing runtimes on the CPU and GPU illustrates the stark contrast in efficiency, further reinforcing the idea that while SYCL offers functional portability, performance portability is not inherently guaranteed. Achieving optimal performance across different hardware platforms always requires architecture-specific tuning, such as selecting appropriate workgroup sizes and optimising memory management strategies. Therefore, although SYCL allows a 'write once, run anywhere' approach, optimising efficiency on various architectures requires additional adjustments tailored to the specific characteristics of the target hardware.

5. Section 5: Comparison of parallelism constructs

On the one hand, we have C+MPI which was designed for programmers with minimal learning curve providing an extension to the existing C programming API. The Message Passing Interface provided developer with constructs that could be applied inline into the program. The shared memory space utilised by MPI proved to be game changer, reducing the need for additional locks and synchronisation as each process carried out individual calculation using its allocated resources. The scaling of MPI across several CPU nodes was also very simple and majorly handled by the mpicc compiler alleviating the need for additional code to provide synchronisation of instructions across CPU nodes.

On the other hand, SYCL is designed for heterogenous systems and introduces an abstraction layer that enables a programming approach for CPUs and GPUs. It provides parallelism constructs such as buffer accessors for immutable variable, `parallel_for` and reduction variables, enabling the implementation of both task and data parallelism. Unlike MPI, SYCL manages memory and communication implicitly, utilizing high-level C++ construct as can be seen on the full grpof analysis (c.f GitLab). Moreover, workgroup size specification in kernel execution allows fine-tuning workloads for specific hardware.

SYCL demonstrates its strength here by significantly reducing execution times for the Euler Totient Sum Algorithm, especially with optimized GPU implementations (*Table 5, 6*). Its portability and usability make it an easy-to-access solution, though achieving peak performance often requires architecture-specific optimisations. While SYCL abstracts much of the complexity seen in MPI, its reliance on backend implementations can lead to variable performance across devices.

(a) MPI implementation challenges

The implementation of the MPI program was relatively simple utilising the foundation provided by euler totient implementation utilising the OpenMP API. There were challenges in implementing the correct source and destinations for sending and receiving messages across processes. During the implementation of the endpoints the correct use of tags was imperative to send the boolean like conditions to terminate all processes other than the master process.

(b) SYCL implementation challenges

The SYCL implementation was simple to set up as we only had to copy the original sequential function in the kernel and adapt them as lambda functions to be used easily. The kernel logic is accessible and highlights the portability of the technology.

6. Appendix A: C+MPI

Gitlab link: [Gitlab](#)

7. Appendix B: SYCL

Gitlab link: [Gitlab](#)